

## 2.38. Grossir un dessin : ×2

Le bouton «  ×2 » fait d'un dessin de 16 une copie de 32, en doublant chaque pixel. À côté, le rond redessiné en 32 : la différence se voit.

### 1. Ce qu'il faut comprendre

**Le zoom d'un dessin.** Dans «  Les tuiles », le bouton  ×2 fait une **copie** du dessin ouvert, deux fois plus grande : un dessin de 8 devient un dessin de 16, un dessin de 16 devient un dessin de 32. L'original ne change pas.

**Comment il grossit :** chaque pixel devient un **carré de 2 × 2** pixels, de la même nuance. Chaque rangée est écrite deux fois, et chaque signe dans la rangée aussi : `"#-"` devient `"##--"`, deux fois.

**À gauche,** `ROND_GRAND` : le rond du 2.01, grossi par ×2. **À droite,** `ROND_32` : le rond redessiné en 32 × 32 (le 2.36). Ils ont la même taille, mais le grossi a des **marches deux fois plus grosses** et un contour de 2 pixels : il n'a pas plus de détails qu'avant, juste de plus gros pixels.

**Quand se servir de ×2 ?** Pour agrandir **ton propre dessin** sans tout recommencer : la copie grossie est un bon point de départ, qu'on affine ensuite pixel par pixel. Pour les formes de la galerie, prends plutôt la taille 32 × 32 : elles y sont redessinées.

**Essaie :** ouvre `perso_ROND_GRAND.cpp` et arrondis ses marches à la main, en t'aidant du rond de droite.

#### Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <sprite32>` → [35.4. La fonction sprite32\(\) — un grand personnage de 32 × 32](#)
- `#include <Perso>` → [35.1. Perso — dessiner des personnages](#)

### 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1712 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      57 o ← la console, à l'allumage — la mise en route de la console
main()         800 o ← démarrage — ta fonction
VBlank         8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte — attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() — écrit un texte, lettre par lettre
RangerLutins   11 o ← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert 15 o ← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles   76 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
Tuiles         640 o ← CopierTuiles — 8 lettres sur 43, et 32 tuiles dessinées
DonneesTransfert 10 o ← InstallerTransfert — la petite routine qui copie les lutins
"GROSSI"       6 o ← texte() — un texte gravé
```

```
"REDESSINE"          9 o ← texte() – un texte gravé
Lettres gravées : DEGINORS
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste,
Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes
Fonctions du langage appelées :
  sprite()   × 32   768 o sur place, dans main()
  texte()    ×  2    22 o sur place, dans main() + EcrireTexte (10 o)
  image()    ×  1     6 o sur place, dans main() + AttendreImage (28 o)
  sprite16() ×  8     0 o sur place, dans main()
  sprite32() ×  2     0 o sur place, dans main()
```

## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- 2.38 Grossir un dessin : x2 ----
// À gauche le rond grossi (chaque pixel doublé), à droite le rond redessiné.

#include <texte>           // texte() : écrire un mot
#include <sprite32>       // sprite32() : un dessin de 32 × 32
#include <Perso>          // Perso : 16 × 16 ou 32 × 32
#include "perso_ROND_GRAND.cpp" // le rond du 2.01, grossi par x2
#include "perso_ROND_32.cpp"  // le rond redessiné en 32 × 32
```

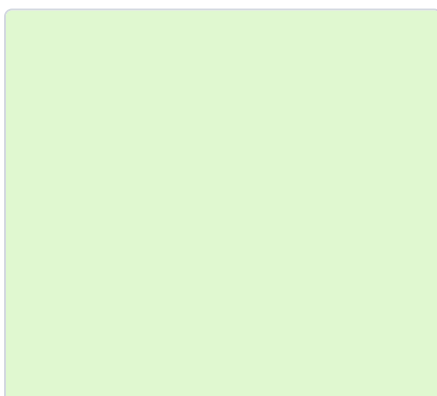
## 2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  sprite32(0, 24, 40, ROND_GRAND); // les lutins 0 à 15
  sprite32(16, 104, 40, ROND_32);  // les lutins 16 à 31
  texte(2, 11, "GROSSI");           // sous celui de gauche
  texte(11, 11, "REDESSINE");      // sous celui de droite

  while (true) {
    image();
  }
}
```

## 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

## 4. Ce qu'on doit voir

Deux grands ronds : à gauche aux grosses marches (GROSSI), à droite plus lisse (REDESSINE).

## 5. Ce que la console répond

---

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le rond grossi en (24, 40)

---

✓ le rond redessiné en (104, 40)

---

✓ les deux mots sont écrits

---

---

Ensuite : **2.39. Réduire un dessin : ÷2** — série 2 : Les formes géométriques. · [gameboy3](#) — écrire une cartouche Game Boy en C++